

@@@@@@@@@' ·G20,000\$ ✓✓✓

```
const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log('MongoDB Connected');
  } catch (error) {
    console.error(error);
    process.exit(1);
  }
};
module.exports = connectDB;

const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'moderator', 'editor', 'admin'], default: 'user' },
  isVerified: { type: Boolean, default: false },
}, { timestamps: true });

userSchema.pre('save', async function(next){
  if(!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.matchPassword = async function(enteredPassword){
  return await bcrypt.compare(enteredPassword, this.password);
}

module.exports = mongoose.model('User', userSchema);

const jwt = require('jsonwebtoken');

const generateToken = (id) => {
```

```

return jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: '30d' });
};

module.exports = generateToken;

const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;
  if(req.headers.authorization && req.headers.authorization.startsWith('Bearer')){
    token = req.headers.authorization.split(' ')[1];
    try{
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      next();
    } catch(error){
      res.status(401).json({ message: 'Not authorized, token failed' });
    }
  } else {
    res.status(401).json({ message: 'Not authorized, no token' });
  }
};

module.exports = { protect };

const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15*60*1000,
  max: 100,
  message: 'Too many requests from this IP, please try again later.'
});

module.exports = limiter;

const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack
  });
};

```

```
module.exports = { errorHandler };
```

```
const User = require('../models/User');  
const generateToken = require('../utils/generateToken');
```

```
const registerUser = async (req, res) => {  
  const { name, email, password } = req.body;  
  const userExists = await User.findOne({ email });  
  if(userExists) return res.status(400).json({ message: 'User already exists' });
```

```
  const user = await User.create({ name, email, password });  
  if(user){  
    res.status(201).json({  
      _id: user._id,  
      name: user.name,  
      email: user.email,  
      role: user.role,  
      token: generateToken(user._id)  
    });  
  } else {  
    res.status(400).json({ message: 'Invalid user data' });  
  }  
};
```

```
const authUser = async (req, res) => {  
  const { email, password } = req.body;  
  const user = await User.findOne({ email });  
  if(user && await user.matchPassword(password)){  
    res.json({  
      _id: user._id,  
      name: user.name,  
      email: user.email,  
      role: user.role,  
      token: generateToken(user._id)  
    });  
  } else {  
    res.status(401).json({ message: 'Invalid email or password' });  
  }  
};
```

```
module.exports = { registerUser, authUser };
```

```
const express = require('express');
```

```
const router = express.Router();
const { registerUser, authUser } = require('../controllers/authController');
```

```
router.post('/register', registerUser);
router.post('/login', authUser);
```

```
module.exports = router;
```

```
const express = require('express');
const dotenv = require('dotenv');
const helmet = require('helmet');
const xss = require('xss-clean');
const hpp = require('hpp');
const cors = require('cors');
const morgan = require('morgan');
const connectDB = require('./config/db');
const limiter = require('./middlewares/rateLimit');
const { errorHandler } = require('./middlewares/errorMiddleware');
const authRoutes = require('./routes/authRoutes');
```

```
dotenv.config();
connectDB();
```

```
const app = express();
app.use(express.json());
app.use(helmet());
app.use(xss());
app.use(hpp());
app.use(cors());
app.use(morgan('dev'));
app.use(limiter);
```

```
app.use('/api/auth', authRoutes);
app.use(errorHandler);
```

```
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

```
const User = require('../models/User');
const generateToken = require('../utils/generateToken');
const crypto = require('crypto');
const nodemailer = require('nodemailer');
```

```
// Register User with Email Verification
```

```

const registerUser = async (req, res) => {
  const { name, email, password } = req.body;
  const userExists = await User.findOne({ email });
  if(userExists) return res.status(400).json({ message: 'User already exists' });

  const verificationToken = crypto.randomBytes(32).toString('hex');
  const user = await User.create({ name, email, password, verificationToken });

  if(user){
    const verificationUrl =
`${process.env.FRONTEND_URL}/verify-email?token=${verificationToken}`;
    const transporter = nodemailer.createTransport({
      service: 'Gmail',
      auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS }
    });
    await transporter.sendMail({
      from: process.env.EMAIL_USER,
      to: user.email,
      subject: 'Email Verification',
      html: `

Click <a href="${verificationUrl}">here</a> to verify your email.</p>`
    });

    res.status(201).json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      message: 'Verification email sent'
    });
  } else {
    res.status(400).json({ message: 'Invalid user data' });
  }
};

// Verify Email
const verifyEmail = async (req, res) => {
  const { token } = req.query;
  const user = await User.findOne({ verificationToken: token });
  if(!user) return res.status(400).json({ message: 'Invalid token' });
  user.isVerified = true;
  user.verificationToken = undefined;
  await user.save();
  res.json({ message: 'Email verified successfully' });
};


```

```

// Login User
const authUser = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if(user && await user.matchPassword(password)){
    if(!user.isVerified) return res.status(401).json({ message: 'Email not verified' });
    res.json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user._id)
    });
  } else {
    res.status(401).json({ message: 'Invalid email or password' });
  }
};

// Forgot Password
const forgotPassword = async (req, res) => {
  const { email } = req.body;
  const user = await User.findOne({ email });
  if(!user) return res.status(404).json({ message: 'User not found' });

  const resetToken = crypto.randomBytes(32).toString('hex');
  user.resetPasswordToken = resetToken;
  user.resetPasswordExpire = Date.now() + 3600000; // 1 hour
  await user.save();

  const resetUrl = `${process.env.FRONTEND_URL}/reset-password?token=${resetToken}`;
  const transporter = nodemailer.createTransport({
    service: 'Gmail',
    auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS }
  });
  await transporter.sendMail({
    from: process.env.EMAIL_USER,
    to: user.email,
    subject: 'Password Reset',
    html: `<p>Click <a href="${resetUrl}">here</a> to reset your password.</p>`
  });

  res.json({ message: 'Password reset email sent' });
};

```

```

// Reset Password
const resetPassword = async (req, res) => {
  const { token, newPassword } = req.body;
  const user = await User.findOne({ resetPasswordToken: token, resetPasswordExpire: { $gt:
Date.now() } });
  if(!user) return res.status(400).json({ message: 'Invalid or expired token' });

  user.password = newPassword;
  user.resetPasswordToken = undefined;
  user.resetPasswordExpire = undefined;
  await user.save();
  res.json({ message: 'Password reset successful' });
};

// Forgot Username
const forgotUsername = async (req, res) => {
  const { email } = req.body;
  const user = await User.findOne({ email });
  if(!user) return res.status(404).json({ message: 'User not found' });

  const transporter = nodemailer.createTransport({
    service: 'Gmail',
    auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS }
  });
  await transporter.sendMail({
    from: process.env.EMAIL_USER,
    to: user.email,
    subject: 'Username Recovery',
    html: `<p>Your username is: ${user.name}</p>`
  });

  res.json({ message: 'Username sent to email' });
};

module.exports = { registerUser, verifyEmail, authUser, forgotPassword, resetPassword,
forgotUsername };

const express = require('express');
const router = express.Router();
const { registerUser, verifyEmail, authUser, forgotPassword, resetPassword, forgotUsername } =
require('../controllers/userController');

router.post('/register', registerUser);

```

```
router.get('/verify-email', verifyEmail);
router.post('/login', authUser);
router.post('/forgot-password', forgotPassword);
router.post('/reset-password', resetPassword);
router.post('/forgot-username', forgotUsername);
```

```
module.exports = router;
```

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
```

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'moderator', 'editor', 'admin'], default: 'user' },
  isVerified: { type: Boolean, default: false },
  verificationToken: { type: String },
  resetPasswordToken: { type: String },
  resetPasswordExpire: { type: Date }
}, { timestamps: true });
```

```
userSchema.pre('save', async function(next){
  if(!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

```
userSchema.methods.matchPassword = async function(enteredPassword){
  return await bcrypt.compare(enteredPassword, this.password);
}
```

```
module.exports = mongoose.model('User', userSchema);
```

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
```

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'moderator', 'editor', 'admin'], default: 'user' },
  subscriptionPlan: { type: String, enum: ['free', 'premium', 'enterprise'], default: 'free' },
```

```
subscriptionExpire: { type: Date },
isVerified: { type: Boolean, default: false },
verificationToken: { type: String },
resetPasswordToken: { type: String },
resetPasswordExpire: { type: Date }
}, { timestamps: true });
```

```
userSchema.pre('save', async function(next){
  if(!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

```
userSchema.methods.matchPassword = async function(enteredPassword){
  return await bcrypt.compare(enteredPassword, this.password);
}
```

```
module.exports = mongoose.model('User', userSchema);
```

```
const User = require('../models/User');
```

```
// تحديث خطة الاشتراك
```

```
const updateSubscription = async (req, res) => {
  const { userId, plan, expireDate } = req.body;
  const user = await User.findById(userId);
  if(!user) return res.status(404).json({ message: 'User not found' });
```

```
  user.subscriptionPlan = plan;
  user.subscriptionExpire = expireDate;
  await user.save();
```

```
  res.json({ message: 'Subscription updated', subscriptionPlan: user.subscriptionPlan,
subscriptionExpire: user.subscriptionExpire });
};
```

```
// تغيير دور المستخدم (Role Management)
```

```
const updateUserRole = async (req, res) => {
  const { userId, role } = req.body;
  const user = await User.findById(userId);
  if(!user) return res.status(404).json({ message: 'User not found' });
```

```
  user.role = role;
  await user.save();
```

```
res.json({ message: 'User role updated', role: user.role });  
};
```

// صفحة إعدادات المستخدم

```
const getUserSettings = async (req, res) => {  
  const user = await User.findById(req.user._id).select('-password -resetPasswordToken  
-verificationToken');  
  if(!user) return res.status(404).json({ message: 'User not found' });
```

```
  res.json({ user });  
};
```

```
const updateUserSettings = async (req, res) => {  
  const user = await User.findById(req.user._id);  
  if(!user) return res.status(404).json({ message: 'User not found' });
```

```
  const { name, email, password } = req.body;  
  if(name) user.name = name;  
  if(email) user.email = email;  
  if(password) user.password = password;
```

```
  await user.save();  
  res.json({ message: 'Settings updated successfully' });  
};
```

```
module.exports = { updateSubscription, updateUserRole, getUserSettings, updateUserSettings  
};
```

```
const express = require('express');  
const router = express.Router();  
const { protect } = require('../middlewares/authMiddleware');  
const { updateSubscription, updateUserRole, getUserSettings, updateUserSettings } =  
require('../controllers/subscriptionController');
```

// فقط Admin - تحديث الاشتراك

```
router.put('/update-subscription', protect, updateSubscription);
```

// فقط Admin - تغيير دور المستخدم

```
router.put('/update-role', protect, updateUserRole);
```

// صفحة إعدادات المستخدم

```
router.get('/settings', protect, getUserSettings);  
router.put('/settings', protect, updateUserSettings);
```

```

module.exports = router;

const subscriptionRoutes = require('./routes/subscriptionRoutes');

app.use('/api/subscription', subscriptionRoutes);

const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const paypal = require('@paypal/checkout-server-sdk');
const User = require('../models/User');

// Stripe Payment
const stripeCheckout = async (req, res) => {
  const { plan, userId } = req.body;
  const user = await User.findById(userId);
  if(!user) return res.status(404).json({ message: 'User not found' });

  const session = await stripe.checkout.sessions.create({
    payment_method_types: ['card'],
    line_items: [{ price: plan, quantity: 1 }],
    mode: 'subscription',
    success_url:
`${process.env.FRONTEND_URL}/success?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.FRONTEND_URL}/cancel`
  });

  res.json({ id: session.id });
};

// PayPal Payment
const paypalCheckout = async (req, res) => {
  const { plan } = req.body;
  const environment = new paypal.core.SandboxEnvironment(process.env.PAYPAL_CLIENT_ID,
process.env.PAYPAL_CLIENT_SECRET);
  const client = new paypal.core.PayPalHttpClient(environment);

  const request = new paypal.orders.OrdersCreateRequest();
  request.prefer("return=representation");
  request.requestBody({
    intent: 'CAPTURE',
    purchase_units: [{ amount: { currency_code: 'USD', value: plan } }]
  });

  const order = await client.execute(request);

```

```
    res.json({ id: order.result.id });  
};
```

```
// Webhook for Stripe
```

```
const stripeWebhook = async (req, res) => {  
    const sig = req.headers['stripe-signature'];  
    let event;  
    try {  
        event = stripe.webhooks.constructEvent(req.body, sig,  
process.env.STRIPE_WEBHOOK_SECRET);  
        if(event.type === 'checkout.session.completed'){  
            const session = event.data.object;  
            const user = await User.findById(session.client_reference_id);  
            if(user){  
                user.subscriptionPlan = session.display_items[0].plan.id;  
                user.subscriptionExpire = new Date(Date.now() + 30*24*60*60*1000); // 30 days  
                await user.save();  
            }  
        }  
        res.json({ received: true });  
    } catch(err){  
        res.status(400).send(`Webhook Error: ${err.message}`);  
    }  
};
```

```
// Referral System
```

```
const applyReferral = async (req, res) => {  
    const { userId, referralCode } = req.body;  
    const referrer = await User.findOne({ referralCode });  
    if(!referrer) return res.status(404).json({ message: 'Referral code invalid' });
```

```
    const user = await User.findById(userId);  
    if(!user) return res.status(404).json({ message: 'User not found' });
```

```
// Reward logic
```

```
    referrer.balance = (referrer.balance || 0) + 10; // example reward  
    await referrer.save();  
    res.json({ message: 'Referral applied successfully', reward: 10 });  
};
```

```
module.exports = { stripeCheckout, paypalCheckout, stripeWebhook, applyReferral };
```

```
const express = require('express');  
const router = express.Router();
```

```

const { protect } = require('../middlewares/authMiddleware');
const { stripeCheckout, paypalCheckout, stripeWebhook, applyReferral } =
require('../controllers/paymentController');

router.post('/stripe-checkout', protect, stripeCheckout);
router.post('/paypal-checkout', protect, paypalCheckout);
router.post('/stripe-webhook', express.raw({type: 'application/json'}), stripeWebhook);
router.post('/apply-referral', protect, applyReferral);

module.exports = router;

const User = require('../models/User');

// Admin Dashboard: Users & Payments Overview
const getAdminDashboard = async (req, res) => {
  const users = await User.find().select('-password');
  const totalUsers = users.length;
  const premiumUsers = users.filter(u => u.subscriptionPlan === 'premium').length;
  const enterpriseUsers = users.filter(u => u.subscriptionPlan === 'enterprise').length;

  res.json({ totalUsers, premiumUsers, enterpriseUsers, users });
};

module.exports = { getAdminDashboard };

const express = require('express');
const router = express.Router();
const { protect } = require('../middlewares/authMiddleware');
const { getAdminDashboard } = require('../controllers/adminController');

router.get('/dashboard', protect, getAdminDashboard);

module.exports = router;

const paymentRoutes = require('../routes/paymentRoutes');
const adminRoutes = require('../routes/adminRoutes');

app.use('/api/payment', paymentRoutes);
app.use('/api/admin', adminRoutes);

const mongoose = require('mongoose');

const aiHistorySchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },

```

```

input: { type: String, required: true },
output: { type: String, required: true },
sentiment: { type: String, enum: ['positive','neutral','negative'], required: true },
createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('AIHistory', aiHistorySchema);

const AIHistory = require('../models/aiModel');
const OpenAI = require('openai');
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

// GPT-5 Response + Sentiment Analysis
const getAIResponse = async (req, res) => {
  const { prompt } = req.body;
  const userId = req.user._id;

  // Generate AI response
  const aiResponse = await openai.chat.completions.create({
    model: 'gpt-5',
    messages: [{ role: 'user', content: prompt }]
  });

  const output = aiResponse.choices[0].message.content;

  // Sentiment Analysis
  const sentimentAnalysis = await openai.chat.completions.create({
    model: 'gpt-5',
    messages: [{ role: 'user', content: `Analyze sentiment of this text: "${output}"` }]
  });

  const sentiment =
    sentimentAnalysis.choices[0].message.content.toLowerCase().includes('positive') ? 'positive' :
    sentimentAnalysis.choices[0].message.content.toLowerCase().includes('negative')
    ? 'negative' : 'neutral';

  // Save History
  const history = await AIHistory.create({ user: userId, input: prompt, output, sentiment });

  res.json({ output, sentiment, historyId: history._id });
};

// Get User AI History
const getAIHistory = async (req, res) => {

```

```

    const history = await AIHistory.find({ user: req.user._id }).sort({ createdAt: -1 });
    res.json({ history });
  };

// Admin: View All AI Interactions
const getAllAIHistory = async (req, res) => {
  const history = await AIHistory.find().populate('user', 'name email role').sort({ createdAt: -1 });
  res.json({ history });
};

module.exports = { getAIResponse, getAIHistory, getAllAIHistory };

const express = require('express');
const router = express.Router();
const { protect } = require('../middlewares/authMiddleware');
const { getAIResponse, getAIHistory, getAllAIHistory } = require('../controllers/aiController');

router.post('/response', protect, getAIResponse);
router.get('/history', protect, getAIHistory);
router.get('/all-history', protect, getAllAIHistory); // Admin only

module.exports = router;

const aiRoutes = require('./routes/aiRoutes');
app.use('/api/ai', aiRoutes);

import React, { useState, useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import AIChat from './pages/AIChat';
import Settings from './pages/Settings';
import AdminPanel from './pages/AdminPanel';
import Navbar from './components/Navbar';
import { ThemeProvider } from './context/ThemeContext';

function App() {
  return (
    <ThemeProvider>
      <Router>

```

```

    <Navbar />
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/dashboard" element={<Dashboard />} />
      <Route path="/ai-chat" element={<AIChat />} />
      <Route path="/settings" element={<Settings />} />
      <Route path="/admin" element={<AdminPanel />} />
    </Routes>
    <ToastContainer position="top-right" autoClose={3000} />
  </Router>
</ThemeProvider>
);
}

export default App;

import React, { createContext, useState, useEffect } from 'react';

export const ThemeContext = createContext();

export const ThemeProvider = ({ children }) => {
  const [darkMode, setDarkMode] = useState(false);

  useEffect(() => {
    const saved = localStorage.getItem('darkMode') === 'true';
    setDarkMode(saved);
  }, []);

  const toggleTheme = () => {
    setDarkMode(prev => {
      localStorage.setItem('darkMode', !prev);
      return !prev;
    });
  };

  return (
    <ThemeContext.Provider value={{ darkMode, toggleTheme }}>
      <div className={darkMode ? 'dark' : ''}>{children}</div>
    </ThemeContext.Provider>
  );
};

```

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { toast } from 'react-toastify';
import { motion } from 'framer-motion';

const AIChat = () => {
  const [prompt, setPrompt] = useState("");
  const [messages, setMessages] = useState([]);

  const sendPrompt = async () => {
    if(!prompt) return toast.error('Please enter a message');
    const res = await axios.post('/api/ai/response', { prompt });
    setMessages([...messages, { input: prompt, output: res.data.output, sentiment:
res.data.sentiment }]);
    setPrompt("");
  };

  return (
    <div className="p-4 max-w-3xl mx-auto">
      <div className="space-y-4">
        {messages.map((msg, i) => (
          <motion.div key={i} initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="p-4 border
rounded shadow">
            <p className="font-bold">You:</p>
            <p>{msg.input}</p>
            <p className="font-bold mt-2">AI ({msg.sentiment}):</p>
            <p>{msg.output}</p>
          </motion.div>
        ))}
      </div>
      <div className="mt-4 flex gap-2">
        <input type="text" className="flex-1 border p-2 rounded" value={prompt}
onChange={(e)=>setPrompt(e.target.value)} placeholder="Type your message"/>
        <button className="bg-blue-500 text-white px-4 rounded hover:bg-blue-600"
onClick={sendPrompt}>Send</button>
      </div>
    </div>
  );
};

export default AIChat;

```

```

import React, { useEffect, useState } from 'react';
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';

```

```

import axios from 'axios';

const Dashboard = () => {
  const [data, setData] = useState([]);

  useEffect(() => {
    const fetchData = async () => {
      const res = await axios.get('/api/admin/dashboard');
      setData([
        { name: 'Total Users', value: res.data.totalUsers },
        { name: 'Premium Users', value: res.data.premiumUsers },
        { name: 'Enterprise Users', value: res.data.enterpriseUsers }
      ]);
    };
    fetchData();
  }, []);

  return (
    <div className="p-4 max-w-4xl mx-auto">
      <h1 className="text-2xl font-bold mb-4">Admin Dashboard</h1>
      <ResponsiveContainer width="100%" height={300}>
        <BarChart data={data}>
          <XAxis dataKey="name" />
          <YAxis />
          <Tooltip />
          <Bar dataKey="value" fill="#3182ce" />
        </BarChart>
      </ResponsiveContainer>
    </div>
  );
};

export default Dashboard;

import React, { useContext, useState } from 'react';
import { Link } from 'react-router-dom';
import { ThemeContext } from '../context/ThemeContext';

const Navbar = () => {
  const { darkMode, toggleTheme } = useContext(ThemeContext);
  const [open, setOpen] = useState(false);

  return (
    <nav className={`bg-gray-100 dark:bg-gray-800 p-4 flex justify-between items-center`}>

```

```

<div className="text-xl font-bold">Advanced App</div>
<div className="hidden md:flex space-x-4">
  <Link to="/">Home</Link>
  <Link to="/dashboard">Dashboard</Link>
  <Link to="/ai-chat">AI Chat</Link>
  <Link to="/settings">Settings</Link>
</div>
<button onClick={toggleTheme} className="ml-4">{darkMode ? 'Light' : 'Dark'}</button>
<button className="md:hidden" onClick={()=>setOpen(!open)}>Menu</button>
{open && <div className="flex flex-col space-y-2 mt-2 md:hidden">
  <Link to="/">Home</Link>
  <Link to="/dashboard">Dashboard</Link>
  <Link to="/ai-chat">AI Chat</Link>
  <Link to="/settings">Settings</Link>
</div>}
</nav>
);
};

```

export default Navbar;

Base Image
FROM node:20-alpine

Set working directory
WORKDIR /app

Copy package.json and install dependencies
COPY package*.json ./
RUN npm install --production

Copy source code
COPY . .

Expose port
EXPOSE 5000

Start application
CMD ["node", "backend/[app.js](#)"]

version: '3.9'

services:

app:
build: .

ports:

- "5000:5000"

environment:

- MONGO_URI=mongodb://mongo:27017/advancedapp
- STRIPE_SECRET_KEY=your_stripe_key
- OPENAI_API_KEY=your_openai_key
- PAYPAL_CLIENT_ID=your_paypal_id
- PAYPAL_CLIENT_SECRET=your_paypal_secret

depends_on:

- mongo

mongo:

image: mongo:6

ports:

- "27017:27017"

volumes:

- mongo-data:/data/db

volumes:

mongo-data:

name: Node.js CI/CD

on:

push:

branches: [main]

jobs:

build-and-deploy:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- name: Set up Node.js
 - uses: actions/setup-node@v3
 - with:
 - node-version: 20
- run: npm install
- run: npm test
- name: Build Docker image
 - run: docker build -t advancedapp .
- name: Push to Docker Hub
 - uses: docker/build-push-action@v4
 - with:
 - push: true
 - tags: yourdockerhubusername/advancedapp:latest

```

const request = require('supertest');
const app = require('../app');
const mongoose = require('mongoose');
const User = require('../models/User');

beforeAll(async () => {
  await mongoose.connect(process.env.MONGO_URI_TEST);
});

afterAll(async () => {
  await mongoose.connection.close();
});

describe('User Endpoints', () => {
  it('should register a new user', async () => {
    const res = await request(app).post('/api/users/register').send({
      name: 'Test User',
      email: 'test@example.com',
      password: '123456'
    });
    expect(res.statusCode).toEqual(201);
    expect(res.body).toHaveProperty('message', 'Verification email sent');
  });

  it('should login a verified user', async () => {
    const user = await User.findOne({ email: 'test@example.com' });
    user.isVerified = true;
    await user.save();
    const res = await request(app).post('/api/users/login').send({
      email: 'test@example.com',
      password: '123456'
    });
    expect(res.statusCode).toEqual(200);
    expect(res.body).toHaveProperty('token');
  });
});

const redis = require('redis');
const client = redis.createClient({ url: process.env.REDIS_URL });
client.connect();

const cache = (key) => async (req, res, next) => {
  const cachedData = await client.get(key);
  if(cachedData) return res.json(JSON.parse(cachedData));

```

```
res.sendResponse = res.json;
res.json = (body) => {
  client.setEx(key, 3600, JSON.stringify(body)); // cache 1 hour
  res.sendResponse(body);
};
next();
};
```

```
module.exports = cache;
```

```
const User = require('../models/User');
```

```
const getApiUsage = async (req, res) => {
  const user = await User.findById(req.user._id);
  res.json({ apiUsage: user.apiUsage || 0 });
};
```

```
const incrementApiUsage = async (userId) => {
  const user = await User.findById(userId);
  user.apiUsage = (user.apiUsage || 0) + 1;
  await user.save();
};
```

```
module.exports = { getApiUsage, incrementApiUsage };
```

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
```

```
import en from './locales/en.json';
import ar from './locales/ar.json';
```

```
i18n
  .use(initReactI18next)
  .init({
    resources: { en: { translation: en }, ar: { translation: ar } },
    lng: 'en',
    fallbackLng: 'en',
    interpolation: { escapeValue: false }
  });
```

```
export default i18n;
```

```
import React from 'react';
import { useTranslation } from 'react-i18next';
```

```

const Home = () => {
  const { t } = useTranslation();
  return <h1 className="text-2xl font-bold">{t('welcome')}</h1>;
};

export default Home;

const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  console.error(`[${new Date().toISOString()}] Error:`, err.stack || err.message);

  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack
  });
};

module.exports = { errorHandler };

const winston = require('winston');
const morgan = require('morgan');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
    new winston.transports.File({ filename: 'logs/combined.log' })
  ]
});

const stream = {
  write: (message) => logger.info(message.trim())
};

const morganMiddleware = morgan('combined', { stream });

module.exports = { logger, morganMiddleware };

const { errorHandler } = require('./middlewares/errorMiddleware');

```

```

const { morganMiddleware } = require('./middlewares/logger');

app.use(morganMiddleware);
app.use(errorHandler);

const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP, please try again later',
  standardHeaders: true,
  legacyHeaders: false
});

module.exports = limiter;

const blockedIPs = ['123.123.123.123', '111.111.111.111']; // Example blocked IPs

const ipBlocker = (req, res, next) => {
  if(blockedIPs.includes(req.ip)){
    return res.status(403).json({ message: 'Access denied from this IP' });
  }
  next();
};

module.exports = ipBlocker;

const limiter = require('./middlewares/rateLimiter');
const ipBlocker = require('./middlewares/ipBlocker');

app.use(limiter);
app.use(ipBlocker);

const mongoose = require('mongoose');

const notificationSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  message: { type: String, required: true },
  type: { type: String, enum: ['info','success','warning','error'], default: 'info' },
  isRead: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
});

```

```

module.exports = mongoose.model('Notification', notificationSchema);

const Notification = require('../models/notificationModel');
const nodemailer = require('nodemailer');

// Create Notification
const createNotification = async (req, res) => {
  const { userId, title, message, type } = req.body;
  const notification = await Notification.create({ user: userId, title, message, type });

  // Send Email
  const transporter = nodemailer.createTransport({
    service: 'Gmail',
    auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS }
  });

  await transporter.sendMail({
    from: process.env.EMAIL_USER,
    to: req.body.email,
    subject: title,
    text: message
  });

  res.json({ message: 'Notification created and email sent', notification });
};

// Get Notifications
const getNotifications = async (req, res) => {
  const notifications = await Notification.find({ user: req.user._id }).sort({ createdAt: -1 });
  res.json({ notifications });
};

// Mark as Read
const markAsRead = async (req, res) => {
  const notification = await Notification.findById(req.params.id);
  if(!notification) return res.status(404).json({ message: 'Notification not found' });

  notification.isRead = true;
  await notification.save();
  res.json({ message: 'Notification marked as read' });
};

module.exports = { createNotification, getNotifications, markAsRead };

```

```

const express = require('express');
const router = express.Router();
const { protect } = require('../middlewares/authMiddleware');
const { createNotification, getNotifications, markAsRead } =
require('../controllers/notificationController');

router.post('/', protect, createNotification);
router.get('/', protect, getNotifications);
router.put('/:id/read', protect, markAsRead);

module.exports = router;

const notificationRoutes = require('../routes/notificationRoutes');
app.use('/api/notifications', notificationRoutes);

import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { motion, AnimatePresence } from 'framer-motion';

const NotificationCenter = () => {
  const [notifications, setNotifications] = useState([]);

  useEffect(() => {
    const fetchNotifications = async () => {
      const res = await axios.get('/api/notifications');
      setNotifications(res.data.notifications);
    };
    fetchNotifications();
  }, []);

  const markRead = async (id) => {
    await axios.put(`/api/notifications/${id}/read`);
    setNotifications(notifications.map(n => n._id === id ? {...n, isRead: true} : n));
  };

  return (
    <div className="fixed top-16 right-4 w-80 space-y-2 z-50">
      <AnimatePresence>
        {notifications.map(n => (
          <motion.div
            key={n._id}
            initial={{ opacity: 0, y: -20 }}
            animate={{ opacity: 1, y: 0 }}

```

```

        exit={{ opacity: 0, y: -20 }}
        className={`p-4 rounded shadow ${n.isRead ? 'bg-gray-200' : 'bg-blue-100'}`}
      >
        <h4 className="font-bold">{n.title}</h4>
        <p>{n.message}</p>
        {n.isRead && <button onClick={()=>markRead(n._id)} className="mt-2 text-sm
text-blue-600">Mark as read</button>}
      </motion.div>
    )))
  </AnimatePresence>
</div>
);
};

export default NotificationCenter;

import React from 'react';
import { motion } from 'framer-motion';

const Modal = ({ isOpen, onClose, children }) => {
  if(!isOpen) return null;

  return (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}
      className="fixed inset-0 bg-black bg-opacity-50 flex justify-center items-center z-50"
    >
      <motion.div
        initial={{ scale: 0.8 }}
        animate={{ scale: 1 }}
        exit={{ scale: 0.8 }}
        className="bg-white dark:bg-gray-800 p-6 rounded shadow max-w-lg w-full"
      >
        {children}
        <button onClick={onClose} className="mt-4 bg-red-500 text-white px-4 py-2 rounded
hover:bg-red-600">Close</button>
      </motion.div>
    </motion.div>
  );
};

export default Modal;

```

```

import React from 'react';
import { Helmet } from 'react-helmet';
import { useTranslation } from 'react-i18next';

const Home = () => {
  const { t } = useTranslation();
  return (
    <>
      <Helmet>
        <title>{t('app_name')} | {t('home')}</title>
        <meta name="description" content="Advanced AI App with GPT-5, multi-user system, and
powerful analytics." />
        <meta name="keywords" content="AI, GPT-5, SaaS, Dashboard, Notifications, Analytics,
Multi-language" />
        <meta name="author" content="Your Company Name" />
      </Helmet>
      <h1 className="text-3xl font-bold">{t('welcome')}</h1>
    </>
  );
};

```

```
export default Home;
```

```

import React from 'react';
import { Helmet } from 'react-helmet';

const LandingPage = () => (
  <>
    <Helmet>
      <title>Advanced AI App | Powerful GPT-5 SaaS Solution</title>
      <meta name="description" content="Experience the ultimate AI-powered SaaS platform with
GPT-5, analytics, multi-language support, and secure user management." />
      <meta name="robots" content="index, follow" />
      <meta name="viewport" content="width=device-width, initial-scale=1" />
    </Helmet>
    <section className="text-center p-10 bg-gradient-to-r from-blue-400 to-indigo-600
text-white">
      <h1 className="text-5xl font-extrabold mb-4">Welcome to Advanced AI App</h1>
      <p className="text-xl mb-6">Your all-in-one SaaS solution with GPT-5 AI, analytics, and
secure team management.</p>
      <a href="/register" className="bg-white text-blue-600 px-6 py-3 rounded font-bold
hover:bg-gray-100">Get Started</a>
    </section>
  </>
);

```

```

</>
);

export default LandingPage;

const User = require('../models/User');
const fs = require('fs');

// Export user data
const exportUserData = async (req, res) => {
  const user = await User.findById(req.user._id);
  const data = JSON.stringify(user, null, 2);
  fs.writeFileSync(`exports/${user._id}.json`, data);
  res.download(`exports/${user._id}.json`);
};

// Import user data
const importUserData = async (req, res) => {
  const file = req.file.path;
  const data = JSON.parse(fs.readFileSync(file));
  await User.findByIdAndUpdate(req.user._id, data);
  res.json({ message: 'Data imported successfully' });
};

module.exports = { exportUserData, importUserData };

const express = require('express');
const router = express.Router();
const { protect } = require('../middlewares/authMiddleware');
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });
const { exportUserData, importUserData } = require('../controllers/dataController');

router.get('/export', protect, exportUserData);
router.post('/import', protect, upload.single('file'), importUserData);

module.exports = router;

const mongoose = require('mongoose');

const teamSchema = new mongoose.Schema({
  name: { type: String, required: true },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  members: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
});

```

```

    createdAt: { type: Date, default: Date.now }
  });

module.exports = mongoose.model('Team', teamSchema);

const Team = require('../models/teamModel');

const createTeam = async (req, res) => {
  const { name } = req.body;
  const team = await Team.create({ name, owner: req.user._id, members: [req.user._id] });
  res.json({ message: 'Team created', team });
};

const addMember = async (req, res) => {
  const { teamId, userId } = req.body;
  const team = await Team.findById(teamId);
  if(!team) return res.status(404).json({ message: 'Team not found' });
  if(!team.members.includes(userId)) team.members.push(userId);
  await team.save();
  res.json({ message: 'Member added', team });
};

module.exports = { createTeam, addMember };

import React, { useEffect, useState } from 'react';
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';
import axios from 'axios';

const AdminAnalytics = () => {
  const [data, setData] = useState([]);

  useEffect(() => {
    const fetchData = async () => {
      const res = await axios.get('/api/admin/analytics');
      setData(res.data.monthlyUsage);
    };
    fetchData();
  }, []);

  return (
    <div className="p-4 max-w-5xl mx-auto">
      <h1 className="text-2xl font-bold mb-4">Monthly Usage Analytics</h1>
      <ResponsiveContainer width="100%" height={400}>
        <LineChart data={data}>

```

```
        <XAxis dataKey="month" />
        <YAxis />
        <Tooltip />
        <Line type="monotone" dataKey="usageCount" stroke="#3182ce" strokeWidth={3}/>
    </LineChart>
</ResponsiveContainer>
</div>
);
};

export default AdminAnalytics;
```